

AVR

Embedded Systems based on AVR Diploma [AVR Diploma]

Partner with us and ignite your students'
passion for technology!

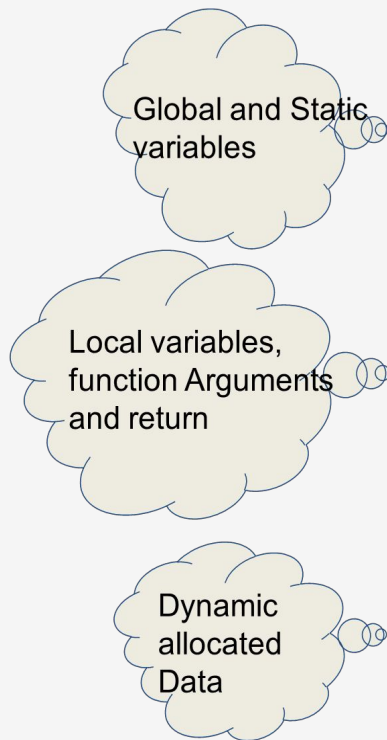


Content

Ram sections	-----●	02
Dynamic Allocation	-----●	03
Malloc	-----●	06
Calloc	-----●	08
Free	-----●	11
Linked List	-----●	13



Basic RAM Sections



RAM

Random
Access
Memory

Need for Dynamic Memory Allocation

Usually so far, the arrays must have fixed length (i.e., length is known at compile time).

```
char arr [10];
```

Length must be defined at compilation time, it can't be a variable ! For that, arrays are statically allocated. But what if we need to create an array with a size defined by the user in the run time ... ?

This is not doable by the normal arrays, but it can be done by Dynamic Allocation.

Dynamic Memory Allocation

We need to use standard library
<stdlib>

function	Description
malloc()	Allocates requested size of bytes and returns a pointer first byte of allocated space
calloc()	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
free()	Deallocate the previously allocated space
realloc()	Change the size of previously allocated space

Dynamic Memory Allocation

We need to use standard library <stdlib>

malloc()

The function malloc() reserves a block of memory of specified size and return a pointer to void which can be casted into pointer of any form.

Syntax :

pointer = (cast_type*) malloc (Size_To_Reserve_In_Bytes)

Example:

```
u8 *ptr ;  
  
ptr = (u8*)malloc(10);
```

Lab 1

Write a C code that apply the bubble sorting algorithm on a set of numbers entered by the user. The code shall ask the user to enter the number of values to be sorted, then the code shall ask the user to enter the values then print them in ascending order.

Expected output

```
Please Enter the number of numbers: 5
Please Enter number 1: 21
Please Enter number 2: 44
Please Enter number 3: 1
Please Enter number 4: 3
Please Enter number 5: 67
Values after sorting are:
1
3
21
44
67
```

Dynamic Memory Allocation

calloc()

The only difference between malloc() and calloc() is that malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size and sets all bytes to zero. The name calloc stands for "contiguous allocation".

Syntax :

pointer = (cast_type*) malloc (number_of_Blocks_To_Resume ,
Size_Of_Block_In_Bytes)

Example:

```
u8 *ptr ;  
  
ptr = (u8*)calloc(10,sizeof(u8));
```

Dynamic Memory Allocation

malloc	<u>calloc</u>
Stands for memory allocation	Stands for contiguous allocation
takes one argument that is, number of bytes.	Take two arguments : number of blocks and size of each block.
Returns a pointer to n bytes of uninitialized storage	returns a pointer to enough free space for an array of n objects of the specified size initialized to zero
malloc is faster than calloc.	<u>calloc</u> takes little longer than malloc because of the extra step of initializing the allocated memory by zero.

Dynamic Memory Allocation

IMPORTANT NOTE

Dynamically allocated memory created with either `calloc()` or `malloc()` doesn't get freed on its own even if it is defined locally in a function. You must explicitly use `free()` to release the space.

Dynamic Memory Allocation

realloc()

If the previously allocated memory is insufficient or more than required, you can change the previously allocated memory size using realloc().

Syntax :

realloc(pointer_to_previously_allocated_block , new_size)

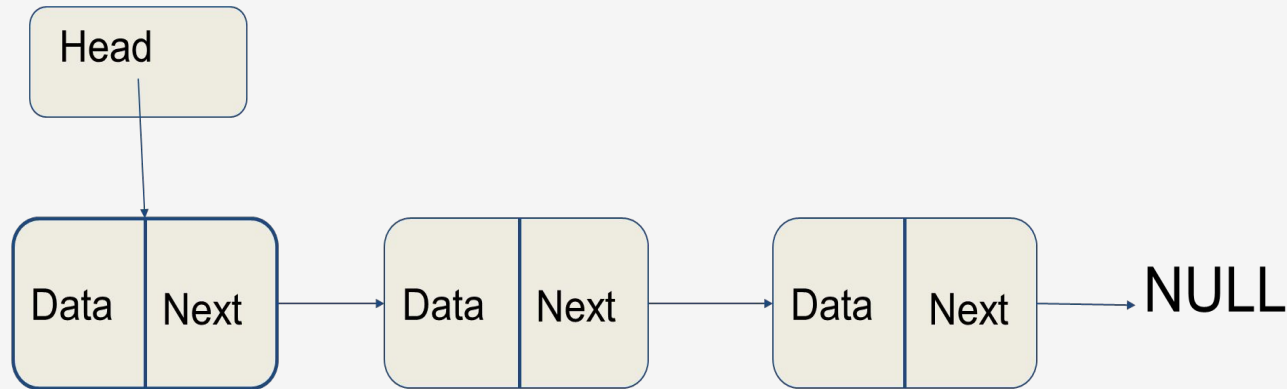
Example:

```
u8 *ptr ;  
  
ptr = (u8*)malloc(10*sizeof(u8));  
  
/*Some code*/  
  
realloc(ptr,20*sizeof(u8));
```

Linked list

Linked list based on construction

The linked list is consisted of some elements each one called a node. The node mainly consists of a data and a pointer to the next node. Keep in mind that each node must points to the next node to keep the list connected.



Linked list

Linked list based on construction

The node simply is a structure has two elements, a data type and a pointer.

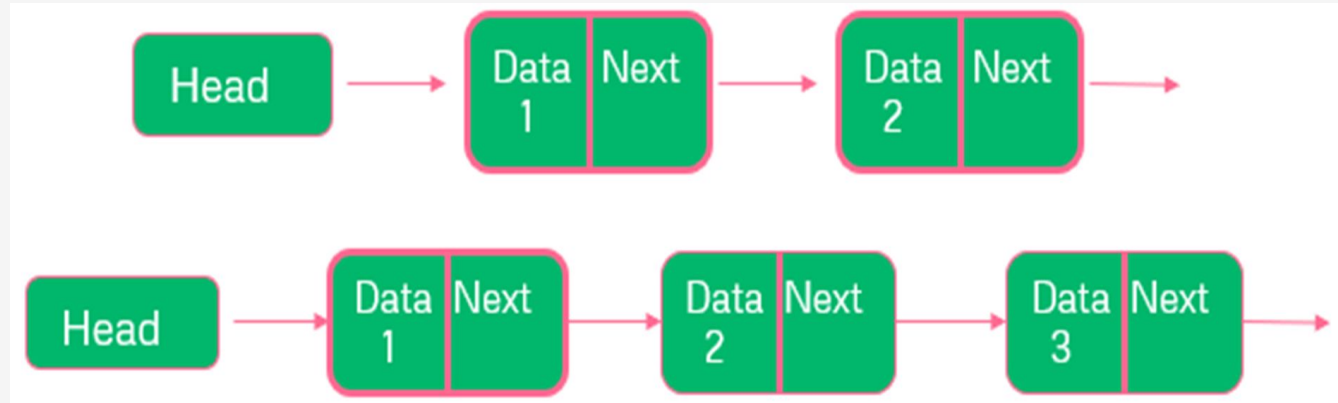
Example:

```
typedef struct Node_type node ;  
  
struct Node_type  
{  
    u32 Data ;  
    node *Next ;  
};
```

Linked list

Linked list based on construction

To add new element to the linked list, you shall create a node dynamically. Assign the data to the node. Then connect the next pointer of the last created node to the new node.



Lab 2

Write a C code that allow the user to add a node to linked list by pressing 0 and to print the linked list by pressing 1. The software shall continue forever till the user enters 2. If the user enters any other numbers the software shall print incorrect entry. See the following scenarios for more details.

Scenario one : Add Node From First

Expected output

```
Please Enter the number of numbers: 5
Please Enter number 1: 21
Please Enter number 2: 44
Please Enter number 3: 1
Please Enter number 4: 3
Please Enter number 5: 67
Values after sorting are:
1
3
21
44
67
```

Lab 2

Write a C code that allow the user to add a node to linked list by pressing 0 and to print the linked list by pressing 1. The software shall continue forever till the user enters 2. If the user enters any other numbers the software shall print incorrect entry. See the following scenarios for more details.

Scenario one : Print Linked List

Expected output

```
Please Enter the number of numbers: 5
Please Enter number 1: 21
Please Enter number 2: 44
Please Enter number 3: 1
Please Enter number 4: 3
Please Enter number 5: 67
Values after sorting are:
1
3
21
44
67
```

Lab 3

Write a C code that allow the user to add a node to linked list by pressing 0 and to print the linked list by pressing 1. The software shall continue forever till the user enters 2. If the user enters any other numbers the software shall print incorrect entry. See the following scenarios for more details.

Scenario one : Add Node From End

Expected output

```
Please Enter the number of numbers: 5
Please Enter number 1: 21
Please Enter number 2: 44
Please Enter number 3: 1
Please Enter number 4: 3
Please Enter number 5: 67
Values after sorting are:
1
3
21
44
67
```

Assignments

Refer to dynamic allocation Assignment





Contact Us



Thank You

Do you have any
questions?